

GUÍA DE TRABAJOS PRÁCTICOS/LABORATORIOS

**TECNICATURA
UNIVERSITARIA EN
PROGRAMACIÓN**
UTN – F.R. Resistencia
RESISTENCIA

- **Asignatura: PROGRAMACIÓN I**
- **Nivel: 1° Año-1° Cuatrimestre**
- **Horario - Carga horaria: 8 horas reloj**

Docente/s:**Comisión 1**

Ing. Blas Cabas Geat (Prof. Adjunto Interino)

Email del docente: blasc147@gmail.com

Ing. Rafael Yurkevich (Jefe de Trabajos Prácticos)

E-mail del docente: rafaelyurk@gmail.com

Comisión 2

Ing. Carolina Ileana Vargas (Prof. Adjunto Interino)

E-mail del docente: civargasar@yahoo.com.ar

TUP Facundo Úferer (Prof. Adjunto Interino)

Email del docente: facundouferer@gmail.com

Comisión 3

Ing. Carolina Ileana Vargas (Prof. Adjunto Interino)

E-mail del docente: civargasar@yahoo.com.ar

TUP Facundo Úferer (Prof. Adjunto Interino)

Email del docente: facundouferer@gmail.com

Coordinador de la Carrera: Ing. Claudia Laclau
tup@frre.utn.edu.ar
www.frre.utn.edu.ar/tup

Reglamento de Estudio: Ord. 1622/2018

Diseño Curricular: Ord 2018/2023

• Trabajo Práctico Unidad 4

- **Unidad Temática:** 4
- **Modalidad:** equipo- individual – presencial.
- **Descripción del Trabajo:**
El trabajo práctico consiste en analizar los problemas propuestos y aplicar los contenidos teóricos de las unidades 1, 2, 3 y 4.
- **Objetivos Específicos:** detallar los objetivos del trabajo en práctico.
 - Resolver problemas mediante estructura de datos estáticas y dinámicas.
 - Identificar los distintos tipos de estructura de datos estáticas y dinámicas.
- **Condiciones y/o actividades previas al desarrollo del TP.**
 - Leer contenido teórico correspondiente a la unidad 4
 - Recuperar los conocimientos de las unidades 1, 2 y 3.
- **Consigna o enunciado de la situación problemática.**

Ejercicio 1: Arreglos

1.1 Dados dos arreglos A y B de números enteros y de 10 posiciones cada uno, se pide:

- Cargar el arreglo A en la declaración de variables.
- Cargar el arreglo B por teclado.
- Generar un arreglo C que contenga la suma de los arreglos A y B.
- Generar un procedimiento para ordenar los arreglos A y B de mayor a menor.
- Generar un arreglo D que contenga los arreglos A y B ordenados de mayor a menos sin elementos repetidos
- Generar una función de búsqueda binaria que dado un valor que se ingresa por teclado en el programa principal devuelva 1 si lo encontró en el arreglo D o 0 si no lo encontró
- Mostrar los tres arreglos por pantalla.

1.2 Realizar un programa que defina un vector llamado “vector_numeros” de 10 enteros, a continuación, lo inicialice con valores aleatorios (del 1 al 10) y posteriormente muestre en pantalla cada elemento del vector junto con su cuadrado y su cubo. Usar la función pow.

1.3 Se quiere realizar un programa que lea por teclado las 5 notas obtenidas por un alumno (comprendidas entre 0 y 10). A continuación debe mostrar todas las notas, la nota media, la nota más alta que ha sacado y la menor.

Ejercicio 2: Arreglos-Búsqueda-Búsqueda binaria

2.1 A partir del arreglo C generado en el ejercicio 1.1, ingresar un valor por teclado e indicar cuantas veces aparece en el arreglo.

Ejercicio 3: Ordenamiento

3.1 A partir de los arreglos A y B del ejercicio 1.1, se pide:

- Generar un arreglo E que contenga ambos arreglos. El arreglo D debe estar ordenado de menor a mayor y no debe contener números repetidos. Usar

un método de ordenamiento diferente al usado en el ejercicio 1. Usar un procedimiento para mostrar por pantalla los tres arreglos.

- Ingresar un número por teclado. Si está comprendido entre el menor y el mayor número de D ,aplicando una función de búsqueda binaria para indicar si se encuentra en el arreglo.

Ejercicio 4: Matrices

4.1 Dada una matriz cuadrada de 3X3, se pide:

- Cargar la matriz con números enteros. La matriz no de contener números repetidos.
- Mostrar la matriz con el siguiente formato

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

- Indicar en que posición se encuentra el máximo valor y el mínimo valor ingresado. Mostrar por pantalla valor y ubicación.
- Generar un arreglo que contenga los elementos de la diagonal principal y de la diagonal secundaria.

4.2 Escribir un programa para resolver el siguiente enunciado

El conjunto $D_{50} = \{1, 2, 5, 10, 25, 50\}$ de los divisores de 50 con las operaciones:

$$a.b = m.c.d.(a,b)$$

m.c.d:máximo común divisor

<u>m.c.d</u>	1	2	5	10	25	50
1						
2						
5						
10						
25						
50						

Ejercicio 5: Arreglos-Registros

5.1. Una empresa de servicios de limpieza necesita almacenar en un arreglo la siguiente información:

- Nro de Cliente
- Tipo de Cliente (E: Empresa – P: Particular)
- Nombre del Cliente
- Nro.Celular de Contacto

Se pide:

- Cargar el arreglo.
- Ordenar el arreglo por Nro. de Cliente en orden ascendente.
- Mostrar por pantalla el listado de clientes ordenados con el siguiente formato

Listado de Clientes

Nro.Cliente	Tipo-Cliente	Nombre	Nro. Contacto

.			
.			
Total de Clientes:			
Total de Clientes Tipo E:			
Total de Clientes Tipo P:			

- Realizar la búsqueda de un cliente por su número. Aplicar búsqueda binaria.
- A partir del arreglo ordenado, generar dos arreglos, uno que contenga los clientes tipo E y otro que contenga los clientes tipo P.

Ejercicio 6: Cadenas

6.1 Escribir un programa en el que se pregunte al usuario por una frase y una letra. Mostrar por pantalla el número de veces que aparece la letra en la frase.

6.2 Escribir un programa que permita obtener la siguiente salida:



Considere que la palabra se ingresa por teclado.

6.3 Escribir el enunciado del problema para el siguiente trozo de programa:

```
char texto[100];
int i=0,n,dif;
dif = 'a'-'A';
printf("Introduzca una cadena: ");
scanf("%s",texto);
while (texto[i] != '\0')
{
    if ((texto[i] >='a') && (texto[i] <='z'))
        texto[i] -= dif;
    else if ((texto[i] >='A') && (texto[i] <='Z'))
        texto[i] += dif;
    i++;
}
texto[i] = '\0';
for (n=0; n<=i; n++)
    printf("%c", texto[n]);
printf("\n\n");
```

Ejercicio 7: Listas-Pilas y Colas

7.1 Listas

7.1.1 Escribir un programa que almacene el abecedario en una lista simplemente encadenada, elimine de esta lista las letras que ocupen posiciones múltiplos de 3, y muestre por pantalla la lista resultante. Preservar la lista original en una lista doblemente encadenada.

7.1.2 Dadas dos listas de enteros positivos cuyos elementos fueron ingresados de menor a mayor, se pide implementar una **función** `MergeListas` que devuelva una lista doblemente encadenada ordenada de menor a mayor con todos los valores de ambas listas.

7.1.3 Implementar una lista que debe permitir almacenar letras, el programa debe incluir las siguientes procedimientos o funciones según corresponda:

- Insertar un objeto al final de la lista.
- Recuperar un elemento de la lista.
- Devuelve el número de objetos en la lista.

7.2 Pilas

7.2.1 Una panadería tiene una lista de facturas de sus clientes. En esta lista están mezcladas las facturas tipo A y B. Para ordenar su administración requiere:

- Armar dos grupos, las facturas tipo A y las facturas tipo B.
- La primera factura a la que se debe acceder es la última emitida generada
- Para ello se cuenta con la siguiente información
 - Nro. Factura
 - Tipo de Factura
 - Nombre del Cliente

7.2.2 Escribir una función *Reemplazar* que tenga como argumentos una pila con tipo de elemento *int* y dos valores *int*: *nuevo* y *viejo* de forma que si el segundo valor aparece en algún lugar de la pila, sea reemplazado por el segundo.

7.2.3 Dada una pila de enteros, escribir una función que determine si sus elementos están ordenados de manera ascendente. Una pila de enteros está ordenada de manera ascendente si, en el sentido que va desde el tope de la pila hacia el resto de elementos, cada elemento es menor al elemento que le sigue. La pila debe quedar en el mismo estado que al invocarse la función. Indicar y justificar el orden del algoritmo propuesto.

7.3 Colas

7.3.1 Se requiere almacenar la información referente a la entrega de proyectos de los alumnos, estos se corregirán por orden de entrega. La información que se requiere registrar es:

- Nro. DNI del alumno
- Nombre y Apellido del alumno
- Link del proyecto

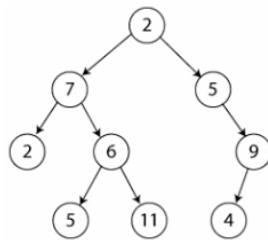
7.3.2 Implementar una función que dada una cola y un número *k*, devuelva los primeros *k* elementos de la cola, en el mismo orden en el que habrían salido de la cola. En caso que la cola tenga menos de *k* elementos. Si hay menos elementos

que k en la cola, devolver un mensaje respecto del tamaño de la cola. Indicar y justificar el orden de ejecución del algoritmo.

7.3.3 Dadas dos colas de enteros positivos sin valores repetidos cuyos elementos fueron ingresados de menor a mayor, se pide implementar una **función** func MergePilas que devuelva un arreglo ordenado de menor a mayor con todos los valores de ambas colas.

Ejercicio 8: Árboles Binarios

8.1 Dado el siguiente árbol binario



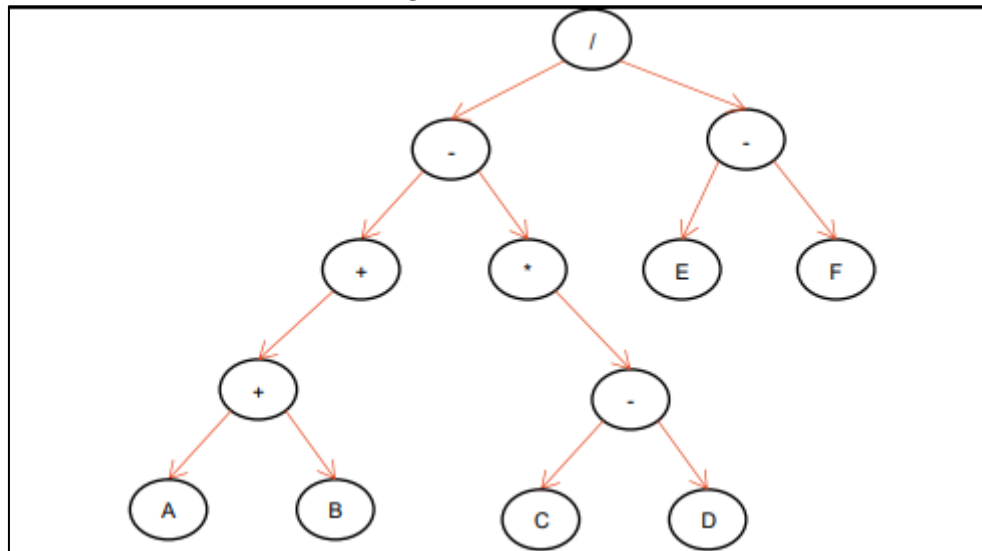
Se pide:

- Recorrer en los 3 modos el árbol y mostrar por pantalla el número obtenido. Implementar cada recorrido mediante funciones, por lo menos una función debe ser recursiva.
- Los resultados obtenidos en cada caso ¿Son iguales?

8.2 Escribe el código C de la función que busca un elemento en un árbol binario.

8.3 Indicar si el árbol del ejercicio 8-1 es perfecto. Se dice que un árbol es perfectamente balanceado, si para cada nodo en el árbol, el número de nodos en sus subárboles izquierdo y derecho difieren a lo más en 1.

8.4 Hallar la función asociada al siguiente árbol ,



Ejercicio 9: Realizar el trabajo de investigación referente a archivos que se encuentra en el Aula Virtual

- **Materiales Necesarios:**
 - Material de estudio de la Unidad 3.
 - Material de estudio de las Unidades 1 y 2.
 - Guía de Actividades Prácticas de la Unidad 3.
 - Software: Lenguaje C.

- **Metodología:** Pasos a seguir para realizar el trabajo práctico.
 - Analizar el problema.
 - Identificar datos de entrada y salida y su tipo de dato.
 - Validación de datos de entrada y manejo de excepciones.
 - Identificar los procesos y operaciones.
 - Identificar la información de salida y el tipo dato.
 - Resolver el problema mediante un algoritmo.
 - Programar el algoritmo en C.
 - Verificar la solución, revisar si la solución es coherente y es correcta.
 - Verificar la aplicación de las buenas prácticas de programación.
- **Resultados Esperados:** Qué se debe obtener al finalizar el trabajo; es decir qué debe “demostrar saber hacer el alumno”.
Que el alumno sea capaz de:
 - Resolver el problema aplicando la estructura de datos estática y/o dinámica óptima para su solución.
 - Verificar si el resultado obtenido es coherente y los pasos realizados son lógicos.
- **Criterios de Evaluación:** Cómo será evaluado el trabajo y los puntos a considerar. La evaluación se realizará del ejercicio 6 mediante un coloquio con el estudiante, en el mismo cual se evaluará:
 - Selección de los tipos de estructura de control e iteración
 - Selección de los tipos de estructura de datos
 - Lógica del Proceso
 - Resultado Obtenido
 - Buenas prácticas de programación

El trabajo práctico tiene un puntaje de 10 puntos y forma parte de la nota de la primera instancia de evaluación.

- **Condiciones para la aprobación.**
 - Entrega del Trabajo Práctico en tiempo y forma.
 - La resolución del Trabajo Práctico es presencial.
 - No se recupera el trabajo práctico.
 - La nota del trabajo práctico es individual.
- **Fecha de Entrega:** Plazo límite para la entrega del trabajo práctico.

Comisión	Fecha
1	26/05/2026
2	26/05/2026
3	27/05/2026

- **Sugerencias de Trabajo**
 - Analizar los datos y la información que se tiene.
 - Si considera que faltan datos y/o información especificar cuáles podrían ser las fuentes para obtenerlas.
 - Asistir a clase.
 - Realizar consultas al docente.

- Leer el material de estudio.
- Resolver los cuestionarios de autoevaluación.

